

Elastic buckling analysis of a cold-formed steel stud column with open-source shell finite element software

Cristopher D. Moen¹, Amoke Shabhari²

Abstract

Shell finite element formulations are employed within a computationally efficient open-source finite element framework to perform elastic buckling analyses of a cold-formed steel stud column. The newly implemented triangular and quadrilateral Mindlin shell finite elements provide accurate solutions to thin-walled buckling problems. An example of an eigenbuckling analysis of a cold-formed steel stud column with service holes will showcase how an open-source software tool can offer improved versatility, accessibility, and computational performance over commercial finite element programs. GitHub-based documentation and validation for the shell elements applied to elastic buckling analysis will be introduced. A web browser-based computational notebook of the example will be available for anyone to run and explore and visualize the buckling modes. The presented paper is instructional in nature and systematically demonstrates the modelling and buckling analysis of a cold-formed steel column using the developed open-source shell finite element analysis procedure, highlighting its potential as a practical tool for cold-formed steel stability analysis.

1. Introduction

Shell finite element analysis is the reference tool for studying the stability of thin-walled cold-formed steel members, and it is the tool of choice whenever a member departs from a prismatic idealization, for example when the web is punched with service holes. Surprisingly few modern shell finite element implementations are available for public inspection, and it is hard to know exactly what the established commercial programs do under the hood. These gaps motivated a multi-year effort to build a trusted, open-source shell finite element capability in the Julia scientific computing language [1], connected to the general-purpose finite element framework Ferrite.jl [2]. Earlier papers described the vision [3], a quadrilateral Mindlin shell element [4], and a triangular Mindlin shell element with plate validation examples [5]. Both elements are now published as Julia packages, TriShellFiniteElement.jl [6] and QuadShellFiniteElement.jl [7], with elastic and geometric stiffness matrices, so that elastic buckling analyses of thin-walled members can be performed with a few hundred lines of open code.

This paper is instructional. It walks through an eigenbuckling analysis of a common building component, a 362S162-33 cold-formed steel wall stud with SFIA service holes [8], and it reports the local, distortional, and global buckling loads and mode shapes obtained with one shell model that uses both elements: quadrilaterals in the structured flats of the stud

and triangles in the unstructured patches around the holes, which is the natural division of labor between the two. Every step is described in the order in which it is coded: cross-section geometry, meshing around the holes, boundary conditions and reference load, pre-buckling stresses, geometric stiffness, the eigenvalue solve, and the identification of the buckling modes. The shell results are compared with finite strip (CUFSM) values for the gross section [9] and with the approximations for members with holes that are used in design [10], [11]. The GitHub-based documentation and validation of the two shell elements are summarized, and a browser-based computational notebook and an interactive mode viewer accompany the paper so that readers can run and explore the example themselves [12].

2. Open-source shell finite elements

2.1 Triangular Mindlin shell element

TriShellFiniteElement.jl implements a flat three-node shell with six degrees of freedom per node, three translations u, v, w and three rotations. The membrane response uses the linear (constant strain) triangle. The bending response follows Mindlin plate theory with the transverse shear treatment of Tessler and Hughes [13]: the rotations are interpolated linearly, the deflection w is interpolated with the three linear functions plus three quadratic mid-side functions, and the three mid-side deflections are eliminated by static condensation so that the element retains only corner nodes.

¹ President and CEO, RunToSolve LLC, cris.moen@runtosolve.com

² Senior Engineer, RunToSolve LLC, amoke.shabhari@runtosolve.com

The condensed transverse shear stiffness is then relaxed by a factor $1/(1 + C_s\alpha)$, where α is the ratio of the shear to bending rotational stiffness traces and $C_s = 0.2$ by default; this relaxation removes the residual shear locking of thin plates and was calibrated so that a simply supported square plate in compression converges to the exact buckling coefficient $k = 4.0$ from above and below within one percent on 16 elements across the plate [5]. A drilling stiffness of the Hughes and Brezzi type [14] ties the in-plane rotation to the skew-symmetric part of the membrane displacement gradient, which is what allows folded plate assemblies such as a lipped channel to be modeled without singular stiffness matrices. The membrane matrix is integrated with one point and the bending and shear matrices with three points, and the element stiffness is rotated from the element plane to global coordinates.

2.2 Quadrilateral Mindlin shell element

QuadShellFiniteElement.jl implements the “4+2” Mindlin quadrilateral described by Moen and Ádány [4], originally written in MATLAB by the second author of that paper. The four bilinear shape functions are supplemented by two bubble functions, $1 - \xi^2$ and $1 - \eta^2$, for the membrane displacements and for the deflection and rotations; the bubble degrees of freedom are condensed out at the element level, which relieves the well-known stiffness of the bilinear quadrilateral in in-plane bending and part of its shear locking. The same shear relaxation as for the triangle is applied with a default $C_s = 0.1$, together with the Hughes and Brezzi drilling term. Membrane terms are integrated with a 2×2 rule and bending and shear terms with a 3×3 rule. The Julia element matrices reproduce the MATLAB reference matrices to machine precision, which is one of the package tests.

2.3 Geometric stiffness and the eigenbuckling problem

For both elements the geometric stiffness matrix is derived from the nonlinear (Green–Lagrange) membrane strains at the mid-surface [15]. It is a function of the membrane stress resultants $N_x = \sigma_x t$, $N_y = \sigma_y t$, and $N_{xy} = \tau_{xy} t$ in the element plane and of the gradients of all three translations, so that in-plane as well as out-of-plane buckling is captured. The elastic buckling problem reads

$$K\varphi = \lambda(-K_g)\varphi, \quad (1)$$

where K is the assembled elastic stiffness, K_g the geometric stiffness assembled from the stresses of a linear static analysis under a reference load, λ the load factor, and φ the buckling mode. With a unit reference load the eigenvalue is directly the elastic buckling load.

2.4 Documentation and validation on GitHub

Each package repository holds a README that documents the element mechanics, the calling sequence with Ferrite.jl,

and the default parameters, together with docstrings on the public functions. The test suites are the validation record. For the triangle they include: element symmetry, rigid body modes, and thickness independence; uniform compression reproducing PL/EA ; a simply supported plate in compression converging to $k = 4.0$ and a clamped plate to $k \approx 10.07$ [16]; invariance of the buckling load to the orientation of the plate in space; and the torsion of flat and folded strips and of a rounded lipped channel reproducing the thin-walled torsion constant within a few percent, which exercises the drilling stiffness. For the quadrilateral the tests add the element-by-element comparison with the MATLAB reference and the plate bending, membrane, and column buckling benchmarks of the earlier paper [4]. Running `Pkg.test()` on either package regenerates every number in this list.

3. Stud column example

3.1 Member, bracing, and holes

The stud is a 362S162-33 lipped channel: web depth 3.625 in (92.1 mm), flange width 1.625 in (41.3 mm), lip length 0.5 in (12.7 mm), and design thickness 0.0346 in (0.879 mm), with $E = 200\,000$ MPa (29 500 ksi) and $\nu = 0.3$. The member is 96 in (2438 mm) long. It is braced at midheight so that the unbraced lengths for flexure about both axes and for torsion are $L_x = L_y = L_t = 48$ in (1219 mm). The ends are pinned and warping free, and the load is uniform compression applied at both ends. Standard SFIA service holes, 1.5 in wide and 4 in long (38.1 mm by 101.6 mm) with semicircular ends, are punched in the web at 24 in (610 mm) on center [8], which places four holes at 12, 36, 60, and 84 in (305, 914, 1524, and 2134 mm) from one end. The brace point is 12 in away from the nearest holes.

3.2 Cross-section polyline

The centerline of the section is built with CrossSectionGeometry.jl from the five flat lengths, the four turning angles, and a centerline corner radius of $2t$, exactly as it would be for a finite strip model. Each flat is subdivided (4 elements on the lips, 20 on the flanges and web) and each corner arc has 4 elements, giving 85 nodes around the section (Figure 1(a)). The same polyline is passed to CUFSM.jl for the finite strip comparison so that the two analyses share their geometry.

3.3 Meshing around the holes

The stud is first extruded as a structured grid: the 85 section nodes are copied onto 489 planes along the length (a longitudinal element length of 5.0 mm, chosen so that nodal planes fall exactly on the four hole centers and on the brace plane). Quadrilateral cells connect neighboring nodes on neighboring planes. Around each hole the structured cells of the web flat within ± 75 mm of the hole center are deleted, and the 92 mm by 150 mm patch is re-meshed in two dimen-

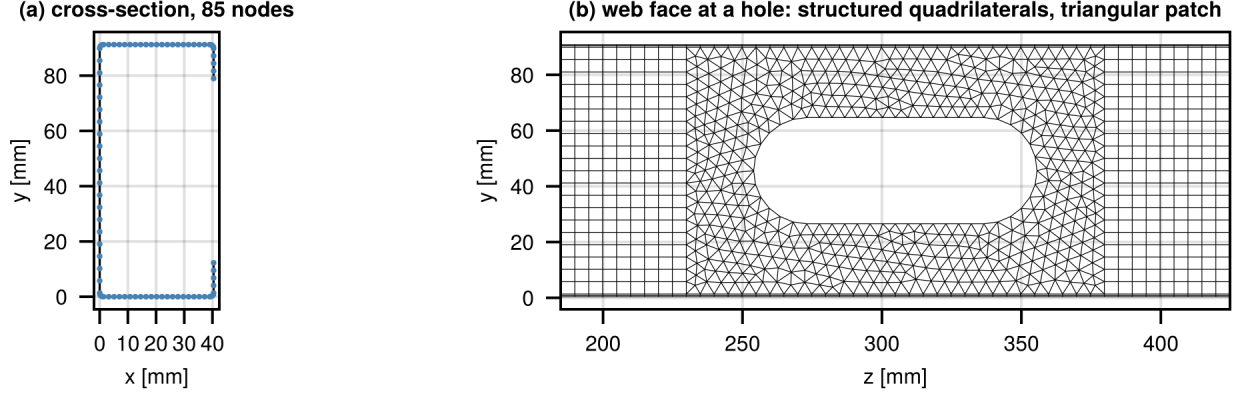


Figure 1: (a) Centerline polyline of the 362S162-33 section, 85 nodes; (b) web mesh around a service hole: structured quadrilaterals outside the 150 mm patch, triangles inside it. Element sizes are 5.0 mm along the length and 4.4 mm across the web.

sions with triangles by the frontal Delaunay algorithm of the Gmsh library [17] with the stadium-shaped hole cut out; triangles fill the irregular region around the hole without the distorted or ill-shaped cells that an all-quadrilateral recombination produces there. Transfinite node counts on the patch boundary equal those of the structured grid, so the patch nodes coincide with the structured nodes and are simply merged. Each semicircular hole end is built from two 90-degree arcs, because a single 180-degree arc is ambiguous in the CAD kernel and was first drawn concave, an error that only a plot of the mesh revealed. One patch is meshed and copied to all four holes. Figure 1(b) shows the result.

The model has 41 333 nodes, 38 592 quadrilaterals, 4 136 triangles and 247 998 degrees of freedom; a refined model with a 2.5 mm longitudinal element length (977 planes) has 509 286 degrees of freedom and serves as the convergence check. Ferrite.jl distributes six degrees of freedom per node through a DofHandler with a translation field u and a rotation field θ ; the two element types live in two SubDofHandlers, and nodes on the patch boundary that belong to both simply share their degrees of freedom:

```
dh = DofHandler(grid)
sq = SubDofHandler(dh, quad_cells); ipq =
  Lagrange{RefQuadrilateral, 1}()
add!(sq, :u, ipq^3); add!(sq, :θ, ipq^3)
st = SubDofHandler(dh, tri_cells); ipt =
  Lagrange{RefTriangle, 1}()
add!(st, :u, ipt^3); add!(st, :θ, ipt^3)
close!(dh)
```

3.4 Boundary conditions and reference load

Pinned, warping-free ends are modeled by fixing the two in-plane translations $u_x = u_y = 0$ at every node of both end sections while leaving the axial translation u_z and all rotations free. The unit reference load $P_{\text{ref}} = 1$ N is applied as equal and opposite axial nodal forces on the two end sections, each node receiving the share of the load proportional

to its tributary centerline length, which produces a uniform compressive stress P/A away from the ends and holes. The midheight brace restrains the translation perpendicular to the web and the twist by fixing $u_x = 0$ at the two web-flange corner nodes of the brace plane, restrains the translation along the web by fixing $u_y = 0$ at the web mid-depth node, and the same node provides the axial datum $u_z = 0$. Because that node lies on the symmetry axis of the section, none of these restraints induces stress in the pre-buckling solution. A practical lesson from setting up this model is that a brace must engage the stiff web-flange corners: when the translations were first restrained at the web mid-depth node only, the thin web simply dimpled around the point restraint and the whole section translated as if unbraced, giving a spurious weak-axis flexural mode at a load far below the braced value.

```
ch = ConstraintHandler(dh)
add!(ch, Dirichlet{:u, end_nodes, (x, t) -> [0.0,
0.0], [1, 2]})
add!(ch, Dirichlet{:u, corner_nodes_at_brace, (x, t)
-> [0.0], [1]})
add!(ch, Dirichlet{:u, Set([web_center_at_brace]),
(x, t) -> [0.0, 0.0], [2, 3]})
close!(ch)
```

3.5 Pre-buckling stresses and geometric stiffness

The elastic stiffness is assembled by one call to each shell package, each working through its own sub-handler and writing into a matrix with the full sparsity pattern, and the two matrices are summed; the static problem is solved with Julia's sparse direct solver, and the membrane stresses are recovered element by element in the element plane (one value per triangle, four Gauss-point values per quadrilateral). Figure 2 shows the longitudinal stress around a hole normalized by P/A : the stress is uniform at 1.0 in the bulk of the member, rises to about 1.9 in the web strips beside the hole and to about 1.25 in the flanges at the hole plane,

and drops to about 0.5 just above and below the hole where the load flows around it. The geometric stiffness is assembled from these stresses, and the constrained degrees of freedom are removed from both matrices.

```
Kq = QuadShellFiniteElement.assemble_global_Ke!
    (allocate_matrix(dh, ch), sq, qr2, qr3, IP4(),
    IP6(), E, v, t; Cs = 0.1)
Kt = TriShellFiniteElement.assemble_global_Ke!
    (allocate_matrix(dh, ch), st, qr1, qr3, IP3(),
    IP6(), E, v, t; Cs = 0.2)
K = Kq + Kt; apply!(K, F, ch); u = K \ F; apply!
    (u, ch)
σ = membrane_stresses(dh, u, E, v, t) # per
sub-handler, element frame
Kg = assemble_Kg(dh, ch, σ, t) #
same pattern: one call per package, summed
```

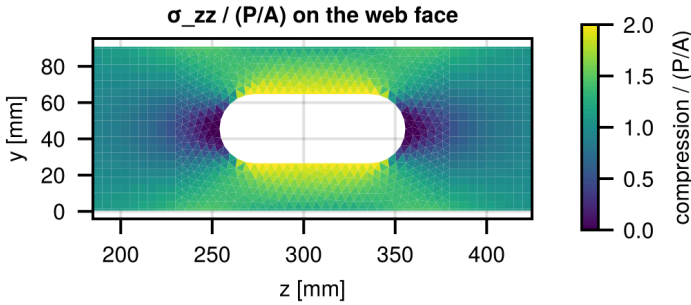


Figure 2: Pre-buckling longitudinal membrane stress on the web face around a service hole, normalized by P/A .

3.6 Solving the eigenvalue problem

The generalized eigenvalue problem of Equation 1 is solved with the implicitly restarted Arnoldi method of ARPACK [18]. The lowest buckling loads are obtained by applying ARPACK to the operator $K^{-1}(-K_g)$, whose largest eigenvalues are the reciprocals of the smallest buckling loads; one sparse Cholesky factorization of K serves all iterations. A perforated member has hundreds of local modes between the first local buckling load and the distortional and global loads, so extracting modes from the bottom of the spectrum would never reach them. Instead, shift-and-invert is used: for a target load σ the operator $(K + \sigma K_g)^{-1}(-K_g)$ returns the modes nearest to σ , and a sequence of targets from 14 to 60 kN sweeps the range of interest, 40 modes per target. Each target requires one indefinite factorization and about 20 seconds on a laptop for the quarter-million degree of freedom model.

```
Kfac = cholesky(Symmetric(K)); tmp =
zeros(ndofs(dh))
op!(y, x) = (mul!(tmp, Kg, x); y .= Kfac \ -tmp)
μ, Φ = eigs(LinearMap(op!, ndofs(dh); ismutating =
true)); nev = 12, which = :LM)
```

```
Pcr = 1 ./ real.(μ) # N, because P_ref = 1
N
```

3.7 Identifying local, distortional, and global modes

Every mode found is classified automatically. A mode whose largest displacement inside the hole patches exceeds twice its largest displacement on the structured planes is localized at the holes. For the other modes the in-plane displacements of each plane are projected onto the three rigid-body motions of the section (two translations and a rotation); a mode whose rigid-body energy fraction exceeds 0.6 is global, and it is labeled weak-axis flexure, strong-axis flexure, and/or torsion according to the dominant rigid-body component on the plane of largest displacement. Otherwise the ratio of the largest displacement at the flange-lip corners to the largest in-plane displacement anywhere and the same ratio at the web-flange corners are compared: a mode in which the flange-lip corners move (ratio above 0.5) while the web-flange corners stay put (ratio below 0.25) is distortional; everything else is local. Half-waves are counted along the lip tip and the web center line, per braced segment. The thresholds were tuned by looking at the modes they select, which the mode viewer makes easy, and the values reported below are the clean representatives of each class (flange-lip ratio above 0.85 for distortional, rigid-body fraction above 0.9 for global, hole ratio above 3 for the hole modes).

4. Results

4.1 Finite strip and closed-form reference values

The CUFSM.jl signature curve of the gross section (Figure 3) has its local minimum at 15.87 kN (3.57 kips) for a half-wavelength of 70 mm and its distortional minimum at 36.58 kN (8.22 kips) at 460 mm; the single half-wave value at 1219 mm is 36.55 kN. The design approximations for the holes follow the procedure used in RunToSolve's SFIA product evaluations with CeeSectionBuckling.jl [19]. For local buckling the net section through the hole is modeled in the finite strip method with a zero-thickness web strip across the 38.1 mm hole width [10]; its local minimum is 24.47 kN at 61 mm, which is above the gross-section value, so the unperforated web governs local buckling of this stud. For distortional buckling the web thickness is reduced to $t_r = t(1 - L_h/L_{crd})^{1/3} = 0.807$ mm following AISI S100 Appendix 2 [11] ($L_h = 101.6$ mm, $L_{crd} = 450$ mm), which lowers the distortional load from 36.67 kN to 34.76 kN (7.82 kips). For global buckling the classical flexural-torsional equation with the gross properties and $KL = 1219$ mm gives $P_{ex} = 309.4$ kN, $P_{ey} = 56.0$ kN, $P_t = 39.3$ kN, and a flexural-torsional load of 37.25 kN (8.38 kips); with the AISI S100 weighted-average net properties (hole length fraction 0.167, warping constant of the net section taken as the sum of the two

halves) the flexural-torsional load drops to 31.10 kN (6.99 kips).

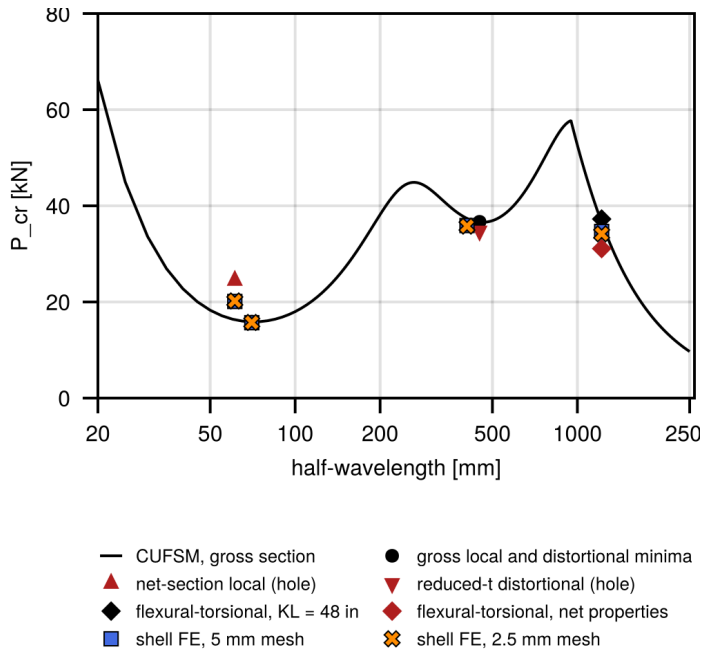


Figure 3: Finite strip signature curve of the gross section with the hole approximations, the analytical global loads for $KL = 48$ in, and the shell finite element results for the perforated stud, plotted at the half-wavelength of the corresponding reference value (global modes at 1219 mm).

4.2 Shell finite element buckling loads

Table 1 collects the characteristic buckling loads of the perforated, braced stud from the 5 mm model and from the refined 2.5 mm model next to the reference values. The lowest buckling load is the gross-section local mode at 15.68 kN (3.53 kips), 15.73 kN with the refined mesh, against 15.87 kN from the finite strip signature curve. Less than one percent of the modal energy of this mode lies in the hole patches; the web buckles between the holes in short half-waves exactly as it would in an unperforated stud, which is what the net-section approximation anticipated when it returned a higher value for the net section than for the gross section. The lowest modes that are concentrated at the holes, in which the two web strips beside a hole bow out of plane while the rest of the member is nearly undeformed, occur at 20.1 kN (20.2 kN refined), below the net-section finite strip value of 24.5 kN. The finite strip idealization applies the average net-section stress to strips that are simply supported along their full length, whereas in the shell model the strips carry up to twice the average stress next to the hole edge (Figure 2) and are only 102 mm long; the stress concentration outweighs the shortness of the strips for this hole geometry. Because the gross-section local mode governs, the discrepancy does not change the design value

of the local buckling load, but it shows that the net-section idealization should not be relied on when the strips beside a hole are the critical elements. The lowest clean distortional mode, with three half-waves in each braced segment (a half-wavelength of about 400 mm) and less than one percent of its energy at the holes, occurs at 35.9 kN (35.8 kN refined); the reduced-web-thickness approximation gives 34.8 kN and the gross section 36.6 to 37.4 kN, so the shell model confirms the hole-adjusted design value within three percent. The refined model also reveals a lower flange-driven mode at 31.8 kN with five half-waves per braced segment (a half-wavelength of about 240 mm), in which the flange rotation of distortional buckling combines with buckling of the web strips at the holes; the 5 mm model finds it at 14 136 kN. This distortional-hole interaction has no finite strip counterpart and is the lowest flange-driven mode of the perforated stud. Below these the classifier also reports modes from about 26 kN upward in which a modest flange rotation accompanies web buckling at the holes; they are listed in the repository but are local modes in the design sense. The lowest global mode is flexural-torsional with one half-wave in each braced segment, at 34.7 kN (34.2 kN refined), which lies between the gross-section analytical value of 37.25 kN and the weighted-average net-section value of 31.10 kN; the holes reduce the torsional stiffness of the stud, but less than the weighted-average approximation assumes. Weak-axis flexure of the braced segments appears higher, at about 46 kN, interacting with distortional deformation of the flanges, against 56.0 kN from the Euler formula for the gross section. Halving the element length changes every characteristic load by less than two percent, which is the convergence evidence for the 5 mm model.

4.3 Mode shapes

Figure 4 shows the lowest local mode around the plane of largest displacement: the web buckles in half-waves of about 70 mm with the flanges rotating slightly, and the holes are bypassed. Figure 5 shows the lowest mode localized at a hole, with the two web strips beside the hole bowing in opposite directions and the flange above the hole participating. Figure 6 and Figure 7 show the lower braced segment of the stud in its lowest distortional and lowest global modes; the brace plane at midheight is a node of both, the upper segment mirrors the lower one, and the global mode combines lateral translation and twist in each 48 in segment. Each figure shows the 5 mm model above the 2.5 mm model; the pictures are nearly indistinguishable, which is the expected outcome for two converged discretizations of the same problem.

Table 1: Elastic buckling loads of the 362S162-33 stud with service holes, kN (kips), from the shell model with 5 mm and with 2.5 mm longitudinal elements, and the finite strip and analytical reference values.

Buckling mode	shell FE, 5 mm mesh kN (kips)	shell FE, 2.5 mm mesh kN (kips)	Finite strip / analytical kN (kips)
Local, lowest	15.68 (3.53)	15.73 (3.54)	15.87 (3.57)
Local, gross section (CeeSectionBuckling)	—	—	15.90 (3.57)
Local at the holes (web strips)	20.10 (4.52)	20.22 (4.55)	24.47 (5.50)
Distortional, 3 half-waves per segment	35.90 (8.07)	35.78 (8.04)	34.76 (7.81)
Distortional, gross	—	—	36.67 (8.24)
Distortional–hole interaction, 5 half-waves per segment	32.09 (7.21)	31.79 (7.15)	—
Global, flexural-torsional	34.67 (7.79)	34.18 (7.68)	37.25 (8.37)
Global, weighted-average net	—	—	31.10 (6.99)
Global, weak-axis flexure	45.96 (10.33)	46.90 (10.54)	55.98 (12.58)

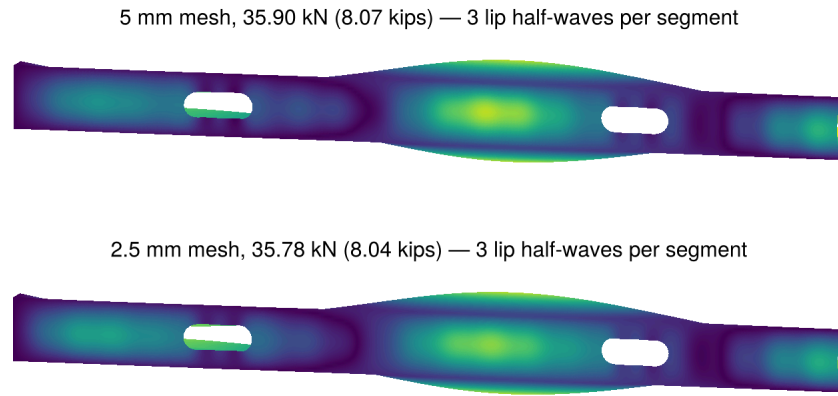


Figure 6: Lowest distortional buckling mode, lower 48 in braced segment, 5 mm mesh (top) and 2.5 mm mesh (bottom).

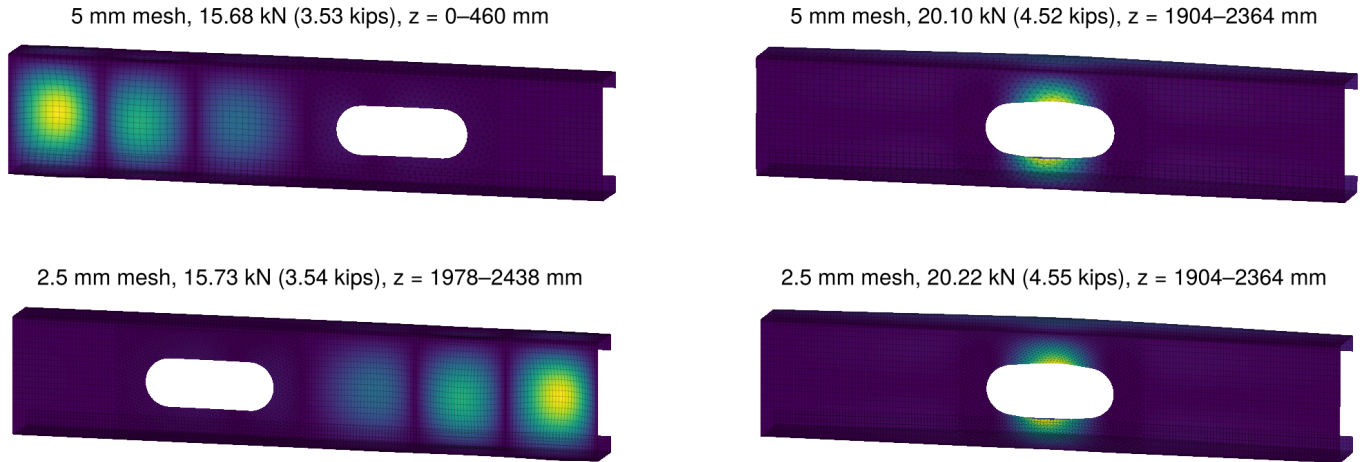


Figure 4: Lowest local buckling mode, close-up around the region of largest displacement, 5 mm mesh (top) and 2.5 mm mesh (bottom).

Figure 5: Lowest buckling mode localized at a service hole (web strips beside the hole), 5 mm mesh (top) and 2.5 mm mesh (bottom).

4.4 Computational performance

Table 2 lists the wall-clock times of the main steps on a 2024 laptop (Apple M3 Max, 14 cores, sparse factorizations on 7 threads) for the quarter-million degree of freedom model and for the refined model with a 2.5 mm longitudinal

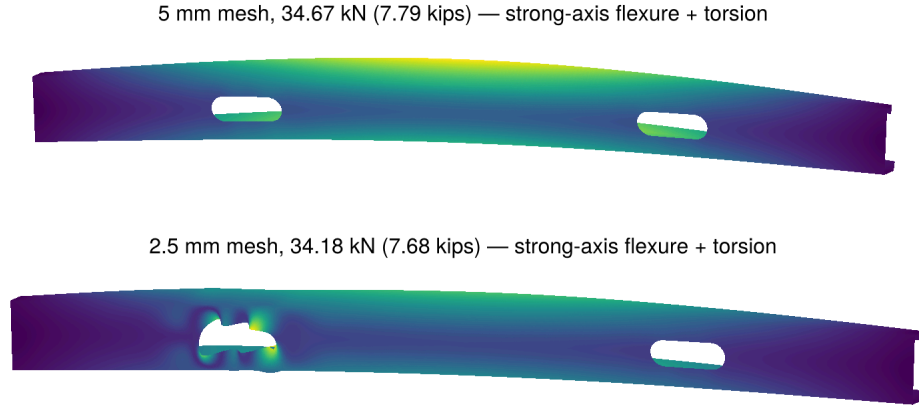


Figure 7: Lowest global (flexural-torsional) buckling mode, lower 48 in braced segment, 5 mm mesh (top) and 2.5 mm mesh (bottom).

Table 2: Wall-clock times in seconds for the eigenbuckling analysis of one model (Apple M3 Max laptop).

Step	5 mm mesh	2.5 mm mesh
Degrees of freedom	247 998	509 286
Assemble elastic stiffness	6	9
Static solve	3	6
Membrane stresses	0	0
Assemble geometric stiffness	2	4
Cholesky factorization of K	9	47
12 lowest modes (ARPACK)	8	29
13 shift-and-invert windows, 40 modes each	284	689

element length. Assembling the elastic stiffness of the 42 728 shell elements takes six seconds, the static solve three seconds, the sparse Cholesky factorization of the stiffness matrix nine seconds, and the twelve lowest buckling modes eight seconds. Each shift-and-invert window of 40 modes costs one indefinite factorization and about 20 seconds, so the full sweep of 13 windows that yields about 200 distinct modes between 16 and 65 kN takes five minutes, and the whole analysis runs in about five minutes with less than 2 GB of memory; the refined model with half a million degrees of freedom takes about two and a half times longer. Julia compiles the packages on first use, which adds roughly a minute to the first run of a session. No part of the procedure is a black box: every matrix is available for inspection, and the analyst can trade the number of shift windows against the completeness of the mode catalogue.

5. Web notebook and mode viewer

Everything in this paper can be reproduced from the companion repository [12], which holds the shared analysis module, the numbered scripts that build the meshes, run the eigenbuckling analyses, compute the reference values, and draw the figures, and the resolved Julia environment. Two entry points are meant for readers rather than for reproduction. A Pluto.jl notebook [20] opens in the web browser and walks through the example cell by cell: the section polyline,

the reference values, the mesh around a hole, an optional live solve of the lowest modes (about one minute on a laptop), and an interactive three-dimensional viewer of the characteristic modes with an amplitude slider. A standalone HTML page exported with Bonito and WGLMakie [21] shows the local, hole, distortional, and global modes of the model as deformed shapes with orbit and zoom controls and needs no Julia installation; it is served from the repository’s GitHub Pages site.

6. Discussion

The example illustrates the three qualities claimed in the abstract. Versatility: the same short script combines two shell element formulations in one model, meshes an arbitrary hole shape by handing a two-dimensional patch to Gmsh, models a brace by naming the nodes it engages, and classifies hundreds of modes with a few lines of linear algebra; each of these would be a menu excursion or a scripting-language detour in a commercial program, and none of them requires a license. Accessibility: the element formulations, their tests, and the whole analysis are public, and the notebook lets a student or a practicing engineer change the section, the hole size, or the bracing and see the consequence within minutes. Performance: a model that resolves local buckling of a 33 mil web with 5 mm shells over a 96 in stud is analyzed in minutes on a laptop, and the

refined model with half a million degrees of freedom takes only about two and a half times longer.

Three lessons of the modeling exercise deserve emphasis. First, a brace must engage the parts of the section that carry its flexural stiffness; restraining a single node on a thin web only dimples the web, and the check that exposed the mistake was a global mode far below the Euler load of the braced segment. Second, the two elements have complementary strengths: the triangle meshes anything but its constant-strain membrane is stiff in the in-plane bending of the lips and flanges that drives distortional buckling, whereas the quadrilateral with its bubble functions converges faster but does not fit around a hole. Using quadrilaterals for the structured flats and confining the triangles to the hole patches takes the best of both, and the refinement check in Table 1 shows that the combination is converged at 5 mm; a model built entirely of 5 mm triangles in an earlier trial overestimated the distortional load by about 5 percent. Third, a perforated member has a dense spectrum of local modes below its distortional and global loads, so the modes of design interest cannot be found by extracting eigenvalues from the bottom of the spectrum; shift-and-invert windows and an automatic classifier that includes a measure of localization at the holes are needed, and both are straightforward to code in this environment.

The comparison with the design approximations is instructive. The reduced-web-thickness rule for distortional buckling is confirmed by the quadrilateral models within two to three percent, and the weighted-average net-section properties are conservative for the flexural-torsional mode of this stud with 4 in holes at 24 in on center. The net-section finite strip idealization, on the other hand, overestimates the buckling load of the web strips beside a hole by about a fifth, because it does not see the stress concentration at the hole edge; for this stud the gross-section local mode governs and nothing is lost, but for a deeper web or a wider hole the strips can govern and the shell model is then the safer reference. None of these conclusions required more than a few minutes of computation, so the shell model can serve both as the reference for such approximations and as a direct design tool when the geometry falls outside their scope.

7. Conclusions

An eigenbuckling analysis of a 96 in cold-formed steel wall stud with standard service holes, braced at mid-height, was carried out step by step with open-source Julia software: the finite element framework Ferrite.jl and the shell element packages TriShellFiniteElement.jl and QuadShellFiniteElement.jl. A quarter-million degree of freedom shell model that combines quadrilateral Mindlin elements in the flats with triangular Mindlin elements around

the holes was built, solved, and post-processed in about five minutes on a laptop, and a refined model confirmed its results.

The local buckling load of the perforated stud equals that of the unperforated section, about 15.8 kN, because the web buckles between the holes. Buckling of the web strips beside the holes occurs at about 20 kN, below the net-section finite strip estimate of 24.5 kN, because the strips carry a concentrated stress next to the hole edge. Distortional buckling with three half-waves per braced segment occurs at about 35.8 kN, in agreement with the reduced-web-thickness approximation within three percent; a lower flange-driven mode at about 31.8 kN, in which distortional rotation of the flanges interacts with buckling of the web strips at the holes, has no finite strip counterpart. The governing global mode is flexural-torsional at about 34.2 kN, between the gross-section and the weighted-average net-section predictions.

Quadrilateral elements in the structured flats and triangular elements in the patches around the holes make one model that meshes the perforated geometry cleanly and converges at a 5 mm element length. A midheight brace must be modeled at the web-flange corners, and shift-and-invert eigenvalue extraction with automatic mode classification is essential for perforated members. The documented and tested element packages, the scripts, a browser notebook, and an interactive mode viewer are publicly available so that the example can be reproduced and extended.

8. Acknowledgments

The authors thank Sándor Ádány for the triangular and quadrilateral shell element formulations and the MATLAB reference implementations, and the Ferrite.jl developers for a finite element framework that made this work possible.

References

- [1] J. Bezanson, A. Edelman, S. Karpinski, and V. B. Shah, “Julia: A fresh approach to numerical computing,” *SIAM Review*, vol. 59, no. 1, pp. 65–98, 2017, doi: [10.1137/141000671](https://doi.org/10.1137/141000671).
- [2] K. Carlsson, F. Ekre, and contributors, “Ferrite.jl: Finite element toolbox for Julia,” *Journal of Open Source Software*, vol. 9, no. 99, p. 6646, 2024, doi: [10.21105/joss.06646](https://doi.org/10.21105/joss.06646).
- [3] C. D. Moen, “A fast, scalable shell finite element formulation implemented with open-source software,” in *Proceedings of the Annual Stability Conference, Structural Stability Research Council*, San Antonio, TX, 2024.
- [4] C. D. Moen and S. Ádány, “Thin shell finite element formulations implemented in open-source software,” in *Proceedings of the Annual Stability Conference, Structural Stability Research Council*, Louisville, KY, 2025.
- [5] C. D. Moen, S. Ádány, and A. Shabhari, “An open-source triangular shell finite element software implementation for the study of thin-walled structures,” in *Proceedings of the Annual*

- Stability Conference, Structural Stability Research Council*, Atlanta, GA, 2026.
- [6] C. D. Moen and A. Shabhari, "TriShellFiniteElement.jl: Mindlin triangular shell finite element formulation for use with Ferrite.jl." [Online]. Available: <https://github.com/runtosolve/TriShellFiniteElement.jl>
- [7] C. D. Moen, "QuadShellFiniteElement.jl: Mindlin quadrilateral shell finite element formulation for use with Ferrite.jl." [Online]. Available: <https://github.com/runtosolve/QuadShellFiniteElement.jl>
- [8] SFIA, "Technical Guide for Cold-Formed Steel Framing Products," Glenview, IL, technical report, 2024.
- [9] B. W. Schafer and S. Ádány, "Buckling analysis of cold-formed steel members using CUFSM: conventional and constrained finite strip methods," in *Proceedings of the 18th International Specialty Conference on Cold-Formed Steel Structures*, Orlando, FL, 2006, pp. 39–54.
- [10] C. D. Moen and B. W. Schafer, "Elastic buckling of cold-formed steel columns and beams with holes," *Engineering Structures*, vol. 31, no. 12, pp. 2812–2824, 2009.
- [11] AISI, "North American Specification for the Design of Cold-Formed Steel Structural Members, AISI S100-24," Washington, DC, technical report, 2024.
- [12] C. D. Moen and A. Shabhari, "ICFSS2026_MoenShabhariStudElasticBuckling: scripts, notebook, and mode viewer for the stud column example." [Online]. Available: https://github.com/runtosolve/ICFSS2026_MoenShabhariStudElasticBuckling
- [13] A. Tessler and T. J. R. Hughes, "A three-node Mindlin plate element with improved transverse shear," *Computer Methods in Applied Mechanics and Engineering*, vol. 50, no. 1, pp. 71–101, 1985.
- [14] T. J. R. Hughes and F. Brezzi, "On drilling degrees of freedom," *Computer Methods in Applied Mechanics and Engineering*, vol. 72, no. 1, pp. 105–121, 1989.
- [15] D. Visy and S. Ádány, "Local elastic and geometric stiffness matrices for the shell element applied in cFEM," *Periodica Polytechnica Civil Engineering*, vol. 61, no. 3, pp. 569–580, 2017.
- [16] S. P. Timoshenko and J. M. Gere, *Theory of Elastic Stability*, 2nd ed. New York: McGraw-Hill, 1961.
- [17] C. Geuzaine and J.-F. Remacle, "Gmsh: A 3-D finite element mesh generator with built-in pre- and post-processing facilities," *International Journal for Numerical Methods in Engineering*, vol. 79, no. 11, pp. 1309–1331, 2009.
- [18] R. B. Lehoucq, D. C. Sorensen, and C. Yang, *ARPACK Users' Guide: Solution of Large-Scale Eigenvalue Problems with Implicitly Restarted Arnoldi Methods*. Philadelphia, PA: SIAM, 1998.
- [19] C. D. Moen, "CeeSectionBuckling.jl: elastic buckling of lipped channel sections with and without web holes." [Online]. Available: <https://github.com/runtosolve/CeeSectionBuckling.jl>
- [20] F. van der Plas, M. Bochenski, and contributors, "Pluto.jl: Simple, reactive programming environment for Julia." [Online]. Available: <https://github.com/fonsp/Pluto.jl>
- [21] S. Danisch and J. Krumbiegel, "Makie.jl: Flexible high-performance data visualization for Julia," *Journal of Open Source Software*, vol. 6, no. 65, p. 3349, 2021, doi: [10.21105/joss.03349](https://doi.org/10.21105/joss.03349).